

FoodETA：外卖配送时效与延迟因素可视分析系统

中期报告

刘鲍非 邓嘉伟 周智洋

2026 年 5 月 31 日

1 项目背景

外卖配送是即时物流中的典型场景。配送时效不仅受到配送距离影响，也会受到天气、交通密度、订单高峰、车辆类型、多单配送和骑手状态等因素共同影响。对于平台运营、配送调度和用户体验分析而言，仅观察平均配送时间不足以解释延迟订单的形成原因。

FoodETA 的目标是对外卖配送时效和延迟因素进行可视分析。项目关注的问题包括：哪些时段更容易出现延迟，天气和交通是否显著增加配送时长，短距离长耗时订单是否存在，骑手评分和车辆类型是否影响配送效率，以及不同城市或区域之间是否存在差异。当前阶段重点是建立可复现的数据处理流程和分析任务框架，为后续可视化系统实现提供稳定数据基础。

2 数据说明

2.1 数据来源与规模

本项目使用 Kaggle 公开数据集 Food Delivery Dataset，数据集链接为：<https://www.kaggle.com/datasets/gauravmalik26/food-delivery-dataset>。

该数据集约包含 45,000 条外卖配送记录，每条记录对应一次配送样本，字段覆盖配送时间、餐厅位置、配送位置、天气、交通、订单类型、车辆类型和骑手属性等信息。

2.2 字段分类

数据字段可分为以下几类。

字段类别	说明
订单基础信息	包括 ID 和 Type_of_order, 用于识别订单样本并比较不同订单类型下的配送表现。清洗后对应 order_id 和 order_type。
配送员属性	包括 Delivery_person_ID、Delivery_person_Age 和 Delivery_person_Ratings, 用于分析骑手特征与配送表现之间的关系。清洗后对应 delivery_person_id、delivery_person_age 和 delivery_person_ratings。
空间位置属性	包括 Restaurant_latitude、Restaurant_longitude、Delivery_location_latitude、Delivery_location_longitude 和 City, 用于计算配送距离并比较城市差异。
时间属性	包括 Order_Date、Time_Orderd 和 Time_Order_picked, 用于分析订单日期、订单小时、取餐时间和时段差异。
环境与交通属性	包括 Weatherconditions、Road_traffic_density 和 Festival, 用于分析天气、交通和节日条件对配送效率的影响。
配送方式属性	包括 Type_of_vehicle、Vehicle_condition 和 multiple_deliveries, 用于比较不同车辆类型、车辆状态和多单配送情况。
配送结果变量	包括 Time_taken(min) 以及预处理生成的 delivery_duration_min、distance_km、is_delayed 和 speed_kmph, 用于衡量配送时效和延迟情况。

2.3 预处理流程

当前仓库已经包含三个数据处理脚本:scripts/preprocess.py、scripts/compute_distance.py 和 scripts/aggregate_data.py。

scripts/preprocess.py 是主入口, 运行方式如下:

```
python3 scripts/preprocess.py
```

该脚本读取 data/raw/food_delivery.csv, 将原始字段统一为 snake_case, 并进一步对齐到清洗后 schema。从 Time_taken(min) 中提取配送时长数字; 当 Time_Orderd 缺失时尝试使用 Time_Order_picked 解析订单小时; 根据 Order_Date 解析星期。

scripts/compute_distance.py 实现经纬度合法性检查和 haversine 距离计算。预处理阶段会删除缺失或明显异常的配送时长、经纬度和距离记录, 并生成清洗后的 data/processed/orders_clean.csv。

`scripts/aggregate_data.py` 负责生成面向后续可视化的数据摘要,输出包括 `overview_summary`、`delivery_time_distribution.json`、`distance_time_sample.json`、`time_period_summary.json`、`hour_summary.json`、`weather_traffic_summary.json`、`courier_vehicle_summary.json`、`city_summary.json`、`risk_scenario_summary.json`、`delay_factor_flow.json` 和 `time_annotations`。

脚本运行时会打印原始记录数、清洗后记录数、删除记录数、主要字段缺失比例、延迟阈值和输出文件列表。

2.4 派生变量

预处理阶段生成以下主要派生变量:

- `delivery_duration_min`: 从 `Time_taken(min)` 中提取的配送时长, 单位为分钟;
- `distance_km`: 使用 haversine 公式计算的餐厅到配送地点距离, 单位为千米;
- `order_hour`: 订单小时, 优先来自 `Time_Orderd`, 缺失时尝试 `Time_Order_picked`;
- `weekday`: 由 `Order_Date` 解析得到的星期;
- `time_period`: 时段标签, 包括 `breakfast`、`lunch_peak`、`afternoon`、`dinner_peak`、`night` 和 `unknown`;
- `is_delayed`: 配送时长是否严格大于全体订单配送时长的 75 分位数阈值;
- `speed_kmph`: 近似配送速度, 异常速度置为空值。

3 分析任务定义

当前项目定义八个核心分析任务,覆盖整体分布、距离关系、外部条件、时间模式、骑手车辆因素、相对延迟、风险场景排序和条件路径流向。

T1 配送时间整体分布

分析 `delivery_duration_min` 的中心趋势、离散程度和长尾延迟情况,判断整体配送效率是否稳定。

T2 配送距离与配送时间关系

基于 `distance_km` 和 `delivery_duration_min` 分析距离与时长的关系,识别短距离长耗时或长距离高效率的异常样本。

T3 天气与交通影响

比较 `weather` 和 `traffic_density` 组合下的平均配送时长、延迟率和订单量,判断外部环境压力。

T4 不同时段效率差异

基于 `order_hour`、`weekday` 和 `time_period` 分析早餐、午高峰、下午、晚高峰和夜间的订单量与配送效率差异。

T5 骑手属性与车辆类型影响

分析 `delivery_person_age`、`delivery_person_ratings`、`vehicle_type` 和 `multiple_deliveries` 对配送表现的影响。

T6 同距离条件下的相对延迟分析

在相近配送距离条件下比较订单是否相对更慢，避免只用全局配送时长阈值造成远距离订单天然更容易被判为延迟。

T7 高延迟风险场景排序与解释

将天气、交通密度、时段、距离区间、车辆类型、多单配送等条件组合成配送场景，构建可解释风险评分，并比较不同场景的订单量、平均配送时长、延迟率、平均距离和多单比例。

T8 配送条件路径与延迟结果流向分析

将 `weather -> traffic_density -> time_period -> is_delayed` 或 `vehicle_type -> multiple_deliveries -> is_delayed` 建模为类别路径，分析不同配送条件组合最终流向延迟或非延迟结果的比例差异。

4 设计方案

系统采用纯前端静态数据架构。数据处理层由 Python、pandas 和 numpy 组成，负责从 `data/raw/food_delivery.csv` 生成清洗后的订单数据和聚合 JSON。前端使用 React、TypeScript、Vite 和 ECharts 构建单页 dashboard，并直接读取仓库中的 `data/processed/*.json` 静态聚合数据。

5 目前实现进展

截至中期阶段，项目已经完成以下内容：

- 明确项目主题和数据集来源；
- 建立 `data/raw/` 与 `data/processed/` 数据目录说明；
- 编写 `requirements.txt`，列出 pandas 和 numpy 等处理依赖；
- 实现 `scripts/compute_distance.py`，包括经纬度合法性检查和 haversine 距离计算；

- 实现 `scripts/preprocess.py`, 包括字段标准化、配送时长解析、经纬度清洗、距离计算、时段派生、延迟标签和速度计算;
- 实现 `scripts/aggregate_data.py`, 输出概览、分布、散点采样、时间、天气交通、骑手车辆、城市、风险场景、因素流向和时间注释聚合 JSON;
- 搭建 React + TypeScript + Vite 前端骨架, 包含 dashboard 布局、基础图表、筛选面板、详情面板和空状态;
- 搭建纯前端数据读取模块, 直接加载 `data/processed/` 下的静态 JSON;
- 完成数据说明、分析任务定义、系统设计和后续工作规划文档。

当前本地已经放置并处理 Kaggle CSV。原始 CSV 通过 `.gitignore` 避免提交到仓库; 处理后的 CSV 和 JSON 已提交到仓库, 便于中期检查和前端直接读取。

6 待完成的工作

后续将基于已经生成的清洗数据和聚合 JSON 继续检查距离、时段和延迟阈值是否符合分析预期。前端将继续使用 React + TypeScript + Vite 和 ECharts 完善主要图表, 并实现多视图联动、筛选器、异常点选择和详情面板。

当前项目保持纯前端实现。数据层将继续围绕 processed JSON 完善: 一方面补充按筛选条件预聚合的静态文件, 另一方面在前端实现局部筛选、分页抽样和 case study 视图所需的数据派生逻辑。重点输出包括 `overview_summary.json`、`delivery_time_distribution.json`、`distance_time_sample.json`、`hour_summary.json`、`weather_traffic_summary.json`、`courier_vehicle_summary.json`、`city_summary.json`、`risk_scenario_summary.json`、`delay_factor_flow.json` 和 `time_annotations.json`。